

Documento de Arquitectura de Software (DAS) - PointCheck

Identificación de Documento

Identificación	
Proyecto	Pointcheck
Versión	1.1
Documento mantenido por	Máximo Zuñiga
Fecha de última revisión	18-04-2026
Fecha de próxima revisión	24-04-2026
Documento aprobado por	Máximo Zuñiga
Fecha de última aprobación	19-04-2026

Historia de Revisiones

Fecha	Versión	Descripción	Autor
19-04-2026	1.1	Arquitectura	Máximo Zuñiga

1. INTRODUCCIÓN

1.1 Contexto del Problema

En el sector de las pequeñas y medianas empresas (PYMES), la gestión de clientes y la organización de turnos suelen realizarse de manera arcaica (manual o presencial). Esto genera la "fricción de salida", donde los usuarios finales perciben una pérdida de tiempo significativa en filas físicas, lo que disminuye la satisfacción y la eficiencia operativa del local.

1.2 Propósito

Este documento proporciona una descripción arquitectónica completa del sistema PointCheck. El objetivo es guiar al equipo de desarrollo en la implementación técnica, asegurando que se cumplan los objetivos de escalabilidad, mantenibilidad y eficiencia definidos para el proyecto académico y su futura proyección comercial.

1.3 Ámbito

PointCheck es una aplicación móvil nativa para Android diseñada para automatizar el agendamiento de horas. El sistema abarca desde la autenticación de usuarios hasta la gestión administrativa y generación de métricas. **Importante:** El sistema no incluye pasarelas de pago integradas; el cobro y la contabilidad se gestionan de forma manual dentro del establecimiento.

1.4 Definiciones, acrónimos y abreviaciones

- **MVVM:** Model-View-ViewModel.
- **Jetpack Compose:** Toolkit moderno para construir UIs nativas en Android.
- **Room:** Capa de abstracción sobre SQLite.
- **Retrofit:** Cliente HTTP para peticiones API.
- **Feature-based Architecture:** Estructura de código organizada por funcionalidades de negocio.

2. VISIÓN DEL SISTEMA

2.1 Descripción General

PointCheck permite a los clientes reservar turnos de atención de forma remota y a los dueños de locales gestionar su flujo de trabajo a través de un panel administrativo con estadísticas en tiempo real.

2.2 Objetivos del Sistema

- Digitalizar el 100% de las reservas manuales de un comercio.
- Reducir los tiempos de espera presencial mediante un sistema de notificaciones y recordatorios.
- Proveer herramientas de análisis (dashboards) para la toma de decisiones gerenciales.

2.3 Requerimientos Funcionales

ID	Nombre	Descripción
RF-01	Autenticación	Registro e inicio de sesión seguro para clientes y administradores.
RF-02	Gestión de Reservas	Creación, cancelación y modificación de citas en horarios disponibles.
RF-03	Dashboard Administrativo	Visualización de flujo de clientes y métricas de atención.
RF-04	Contabilidad Manual	Módulo para registrar el cierre de ventas realizadas físicamente en el local.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1 Patrón: MVVM (Feature-based)

Se ha seleccionado una arquitectura **Feature-based MVVM**. A diferencia de Clean Architecture tradicional, este enfoque organiza el código por funcionalidades (ej. Auth, Booking, Dashboard) en lugar de por capas técnicas (ej. Data, Domain, Presentation), aunque mantiene estas subdivisiones internamente cuando la complejidad lo requiere.

3.2 Justificación del Estilo

- **Escalabilidad:** Permite trabajar en módulos independientes sin afectar al resto del sistema.
- **Simplicidad (Selective Domain):** La capa *Domain* solo se implementa en funcionalidades con lógica de negocio compleja para evitar el exceso de archivos innecesarios.
- **Modernidad:** Alineado con las recomendaciones actuales de Google para el desarrollo nativo en Kotlin y Compose.

4. MODELO DE VISTAS (VISTA LÓGICA)

4.1 Estructura de Paquetes

Basado en el repositorio GitHub oficial (ismaeljim/PointCheck), la estructura se define como:

```
com.pointcheck.app
├── core/
│   ├── database/ (Room implementation)
│   ├── network/ (Retrofit interfaces)
│   ├── navigation/ (Compose Navigation setup)
│   └── notifications/ (Push setup)
├── features/
│   ├── auth/ (Login, Register)
│   ├── booking/ (Calendar, Reservation flows)
│   └── dashboard/ (Admin metrics, Billing stats)
```

5. REQUISITOS DE CALIDAD

Atributo	Descripción	Métrica de Éxito
Rendimiento	La aplicación debe responder rápidamente a las interacciones.	Carga de pantallas en menos de 1.5 segundos.
Seguridad	Protección de datos sensibles de usuarios.	Uso de cifrado en DataStore y HTTPS.
Mantenibilidad	Facilidad para corregir errores o añadir funciones.	Separación clara de responsabilidades en el ViewModel.

6. PRINCIPIOS DE DISEÑO APLICADOS

- **Cohesión:** Cada módulo de funcionalidad tiene una responsabilidad única.
- **Bajo Acoplamiento:** Los componentes de UI no conocen la fuente de los datos (Network vs Local Database).
- **Single Source of Truth:** El Repositorio es la única fuente de verdad para la UI.

7. EQUIPO Y CONTROL DE VERSIONES

El proyecto es gestionado mediante Git bajo una estrategia de *Feature Branching*.

- **Director de Proyecto:** Máximo Zúñiga.
- **Desarrollador:** Ismael Jimenez.
- **Diseño UI/UX:** Benjamín Devia.